queries on Big Data. As data volumes have been growing exponentially, there has been an increasing amount of continuing research happening in this space to create computing models that reduce latency. Apache Spark was an effort in that direction, which worked on reducing latency using in-memory data structures.

Blink DB went further to squeeze the latency benchmarks by driving a notion of approximate queries. There were several industry use cases that appeared to tolerate some error in the answers, provided it was faster. BlinkDB does this by running queries against samples of original datasets rather than the entire datasets. The framework makes it possible to define the acceptable error bounds for queries, or specify a time constraint. The system processes the query based on these constraints and returns results within the given bounds.

BlinkDB leverages the notion of the statistical property – 'sampling error'– which does not vary with the population size, but rather depends on the sample size. So the same sample size should hold reasonably well with increasing data sizes. This insight leads to an incredible improvement in performance. As the time taken in query processing is mostly I/O bound, the processing time can be increased by as much as 10x with a sample size of 10 per cent of the original data, with an error of less than 0.02 per cent.

BlinkDB is built on the Hive Query engine, and supports both Hadoop MR as well as Apache Shark execution engines. BlinkDB provides an interface that abstracts the complexities of approximation, and provides an SQL-like construct with support for standard aggregates, filters and groupBy, joins and nested queries, apart from user-defined functions with machine language primitives.

Cluster resource management frameworks

Cluster resource management is one of the key components in the Big Data processing stack. Successful frameworks that have emerged have been able to combine generality in supporting different frameworks with disparate processing requirements as well as the robustness to handle volume and recovery seamlessly. A generic framework will avoid the need to replicate massive amounts of data between different disparate frameworks within a cluster. It is also very important to provide interfaces at this layer that enable ease of administration and use. We will discuss a couple of frameworks that have shown promise at this layer.

Apache Hadoop Yarn *(https://hadoop.apache.org/)*: Hadoop 1.0 was written solely as an engine for the MapReduce paradigm. As Hadoop became widely accepted as a platform for distributed Big Data batch processing systems, requirements grew for other computing patterns like message passing interfaces, graph processing, real-time stream processing, ad hoc and iterative processing, etc. MapReduce, as a programming pattern, did not support these kinds of requirements, and newer (as well as other existing) frameworks